AI-BERT VIBE CODING · ALBERT SCHOOL

03 · Git as a safety net

le projet est sur GitHub avec un historique, un retour arrière visible, et Cloudflare redéploie tout seul à chaque push.

How to use this module

Work through it alone, at your own pace, in about an hour. Everything you need is here: the idea, the exact commands, the prompts to copy, and what to do when it breaks. Bring your questions to the weekly help hour. The last section is for the organisers.

The idea

The version from two minutes ago is nowhere

You asked for a big redesign, the agent rewrote every file, and the result is broken. Undo does nothing, because these are not keystrokes, they are files that were replaced. Claude Code keeps checkpoints while its window is open, and they vanish when you close it. Without a safety net that lives in the project itself, the only way back is memory.

A commit is a named save point

Exactly like saving a game before a boss. You save, you try the risky thing, and if it goes wrong you load the save. You make one before anything risky, which with an agent means constantly. A commit has a message, so your future self knows what "before" was.

Going back is a command, not a drama

- `git restore .` throws away everything you changed since the last save.
- `git revert` undoes a save you already made, and records that you did.

Nothing is ever lost. That is the contract, and it changes how brave you can be with an agent.

Push means: send your saves to GitHub

A commit lives on your laptop. Push copies it to GitHub. Pull brings back what is there. Three words, that is all the vocabulary tonight needs.

GitHub is the copy elsewhere, and your portfolio

Your laptop can die tonight. GitHub keeps the history somewhere else. And a recruiter opening your profile sees dated commits with real messages, which is evidence, not a claim.

The loop gets shorter tonight

You describe -> the agent writes -> you commit -> you push -> Cloudflare deploys by itself.

No more drag and drop. Save, push, done. From now on every step of this path ends with a push.

Let the agent run git, but read what it does

"Commit this with a clear message" works. So does "undo the last change but keep the title". Always read the commands it proposes before saying yes. Git will not delete your work, but you should still know what it is doing with it.

Prompts for this module

Replace everything between < and > with your own words before you press Enter. The agent takes placeholders literally.

01 First save

```
Turn this folder into a git repository on a branch called main and make a first commit with the message "Page after step 02". Show me the commands before running them, then run them and show me git log.
```

02 Break it on purpose

```
Redesign the whole page with a dark theme and a menu. Do it, I want to see it broken.
```

03 Get back

```
Get me back to the last commit. Show me the command before running it, and tell me whether it will show in the history or not.
```

04 Push to GitHub

```
I created an empty repository at <url>. Connect this folder to it and push. Show me each command before running it. If authentication fails, tell me exactly what to type.
```

05 A real change, saved

```
Change <one precise thing>. Then commit it with a message that says what changed, in plain words, and push.
```

What you build

What you have at 00:57

Your project on GitHub with at least two commits and a readable message on each, and one visible undo.

Where to start

Open a terminal in your step 02 folder, type `claude`. Your first prompt turns the folder into a repository and makes the first save.

You did the three "before step 03" items on the Setup page, at home. If not, sit with your pair tonight.

No step 02 folder? Download the step 01 starter, run its first prompt, then continue here.

Constraints

- **Save before you break.** Your first commit happens before any change tonight.
- **Break it on purpose once**, then get back. Either commit the breakage and `git revert` it, so the undo shows in the history, or `git restore .` and paste the command in `notes.md`.

Bonus, if you finish early

Connect Cloudflare to the repository: Create -> Pages -> Connect to Git -> a **new** project (your old name is taken, your URL changes tonight, that is fine). Build command empty, output directory `/`. Then push once and watch it deploy itself.

If you get stuck

1. `fatal: not a git repository` -> your terminal is in the wrong folder. `cd` into the project folder and try again.
2. **Please tell me who you are** -> run `git config --global user.name "Your Name"` then the same with `user.email`.
3. `git push` **is refused** -> GitHub does not know your laptop yet. Run `gh auth login` and follow the prompts, then push again.
4. Raise your hand.

Done means

Every line is something another person can check without reading your code. Tick them as you go.

- [ ] My repository is on GitHub with at least two commits and a readable message on each
- [ ] One undo is visible: a revert in the history, or the restore command in notes.md
- [ ] I can say in one sentence what a commit is
- [ ] I posted the repository link in the club group
- [ ] **Bonus.** Cloudflare is connected to the repository and deployed my last push by itself

How it was done

> **Read this after you have tried, not before.**
> The value of this module is in the attempt, not in the answer. If you are stuck for more than twenty minutes, read on.

What a good result looks like

A GitHub repository with a readable history: "Page after step 02", "Add a dark theme", "Revert dark theme". Two commits and an undo is a full score. Cloudflare connected to the repository is the bonus.

The exact steps, for the record

1. `git init -b main`, `git add .`, `git commit -m "Page after step 02"`. If git asks who you are, set your name and email once with `git config --global`.
2. Break something big. `git restore .` to get back, or commit it and `git revert HEAD`.
3. Create an empty repository on GitHub, then `git remote add origin <url>` and `git push -u origin main`.
4. Bonus: Cloudflare -> Create -> Pages -> Connect to Git -> a new project on your repository. No build command, output directory `/`.
5. Bonus: change one word, commit, push. Watch the deployment happen without you.

The two mistakes almost everyone made

Committing after breaking instead of before. The save has to exist before the risk, or it saves the broken state.

Committing everything as "update". In two weeks you will not know which save is which. Say what changed.

Why the agent made this easier, not harder

It ran the commands for you. But the ones who read the commands before saying yes were the ones who could explain their history at the end. Those are the Ownership points, in every contest from next week on.

Pour les organisateurs

Cette dernière partie est en français et ne s'adresse pas aux membres. C'est le conducteur de l'heure d'entraide et la préparation à faire avant.

S03 — Git as a safety net · conducteur

Victoire de fin d'heure : le projet est sur GitHub avec un historique, un retour arrière visible, et Cloudflare redéploie tout seul à chaque push.

Notion unique : un commit est un point de sauvegarde. Branche et revert en corollaire. Pas de théorie git au-delà.

Avant la séance

- Ouvrir la séance 3 dans l'admin.
- Compte GitHub de démo prêt, dépôt vide `demo-s03` créé d'avance.
- Un projet Cloudflare Pages de démo à connecter à ce dépôt.
- Vérifier que `git --version` répond sur la machine de démo.
- Contrôle à l'entrée : `gh auth status` chez chaque étudiant. Ceux qui n'ont pas fait le devoir « avant la séance 3 » rejoignent un binôme.
- Pratique réaliste : 30 à 35 min. La DoD s'arrête aux commits et à l'annulation ; Cloudflare est en bonus.

00:00–00:03 · Accueil

« Ce soir, on rend impossible de perdre son travail. À 00:60, casser votre site n'aura plus aucune importance. »

00:03–00:08 · L'échec

1. Sur la page S02, prompt : « Redesign the whole page with a dark theme, a menu and three sections ». Laisser l'agent réécrire tout.
2. Ouvrir. C'est cassé, ou moche, ou les deux. « Je veux revenir à avant. » Cmd+Z ne fait rien, les fichiers ont été réécrits.
3. Silence. « Où est la version d'il y a deux minutes ? Nulle part. »
4. Si quelqu'un cite `/rewind` ou double Échap : le montrer, puis fermer le terminal et rouvrir. Plus de checkpoint. Le filet de Claude vit dans la session ; git vit dans le projet, sur GitHub, et redéploie.

00:08–00:20 · Le concept, trois affirmations

1. **Un commit est un point de sauvegarde nommé.** Comme une sauvegarde de jeu. Tu en fais un avant chaque chose risquée, donc tout le temps.
2. **Revenir en arrière est une commande, pas un drame.** `git restore` pour annuler ce qui n'est pas encore sauvegardé, `git revert` pour annuler une sauvegarde. Rien n'est jamais perdu, c'est le contrat.
3. **GitHub est la copie ailleurs, et ton portfolio.** Ton disque peut mourir. Et un recruteur qui ouvre ton profil voit des commits datés, pas des promesses.

Au tableau : fichiers -> commit -> commit -> commit, une flèche « revert » qui remonte. Puis une flèche vers GitHub, puis GitHub -> Cloudflare.

00:20–00:28 · La démo

1. Dans le dossier S02 : `git init -b main`, `git add .`, `git commit -m "Page after step 02"`. Montrer `git log`.

2. Refaire le redesign destructeur. Constater. `git restore .` Tout revient. Applaudissements attendus.
3. Refaire un vrai petit changement, commit. `git push` vers le dépôt GitHub. Montrer l'historique sur le site GitHub.
4. Cloudflare : Create -> Pages -> Connect to Git -> un NOUVEAU projet, l'ancien nom est pris, l'URL change ce soir. Aucune commande de build, output `/`. Attendre. Changer un mot, commit, push : le site se met à jour seul. C'est le bonus des étudiants, pas leur DoD.

00:28–00:30 · Lancement

Énoncé projeté. Prévenir : l'agent peut faire les commandes git pour vous, mais lisez ce qu'il fait.

00:30–00:57 · Pratique

Pièges : dossier ouvert au mauvais endroit, push refusé faute de connexion GitHub, oubli de `git add`. Les trois sont sur l'énoncé. À 00:45, deux volontaires.

00:57–01:00 · Vitrine

Montrer un historique GitHub avec un revert dedans. Demander : qu'est-ce qui vous a fait peur avant et plus maintenant ?

Après

Corrigé déverrouillé. RETEX. Le concours 1 est dans huit jours : l'annoncer.